PLANO DE TESTES DE QUALIDADE

Aplicações desktop QT + servidor Flask

12 aplicações • testes de interface, dados, integração e design patterns

Objetivo e critérios de execução

Este documento propõe cenários funcionais e de qualidade para aplicações Python com cliente desktop em QT e servidor Flask. Cada aplicação tem quatro testes reproduzíveis, com pré-condições, ações e resultados esperados; adaptar nomes de telas e endpoints aos contratos implementados.

Critério obrigatório de arquitetura: cada aplicação deve demonstrar, por comportamento observável e não apenas por presença de classes, pelo menos um padrão criacional, um estrutural e um comportamental. Os padrões indicados em cada seção são sugestões compatíveis com o domínio e devem ser substituídos se a solução do projeto adotar outros equivalentes.

Como executar e avaliar

– Interface QT: executar ações pelos controles visíveis, conferir estados, mensagens, navegação, atualização da tela e ausência de travamentos.

– Dados: testar entradas válidas, ausentes, fora de faixa, malformadas, duplicadas e limites de negócio; verificar persistência e ausência de gravações parciais.

– Cliente-servidor: validar requisição/resposta Flask, códigos de erro, autenticação/autorização, consistência após nova consulta e comportamento em falha de rede.

– Design patterns: observar que a responsabilidade do padrão está isolada e que o resultado permanece correto nos casos válidos e inválidos.

– Critério de aprovação sugerido: todos os resultados esperados atendidos, nenhum dado indevido persistido, erros recuperáveis apresentados com clareza e nenhum defeito crítico de autorização ou duplicidade.

Ambiente recomendado

Usar banco de teste isolado, contas com papéis distintos, dados reinicializáveis e um servidor Flask de teste. Para cenários de integração, registrar requisições/respostas e limpar os dados ao final; nos casos de erro, conferir tanto a mensagem exibida no QT quanto o estado persistido no servidor.

1. Chat online

Design patterns a demonstrar

– Criacional — Factory Method: criar usuários/mensagens conforme o tipo.
– Estrutural — Proxy: controlar acesso ao canal e validar identidade/permissão.
– Comportamental — Observer: distribuir mensagens aos clientes inscritos.

Cenários de teste

T1.1 — Envio e atualização da conversa

T1.2 — Mensagem vazia e limite de tamanho

T1.3 — Falha de conexão e reenvio

T1.4 — Observer e Factory Method

2. Calendário de atividades

Design patterns a demonstrar

– Criacional — Builder: construir atividade com campos obrigatórios e opcionais.
– Estrutural — Facade: simplificar criação, edição e consulta de eventos.
– Comportamental — Observer: atualizar calendário e lembretes após alterações.

Cenários de teste

T2.1 — Criar e exibir atividade

T2.2 — Validar período e campos obrigatórios

T2.3 — Persistência e atualização entre clientes

T2.4 — Builder e Observer

3. Mural de Kanban

Design patterns a demonstrar

– Criacional — Factory Method: criar cartões por tipo (tarefa, bug, melhoria).
– Estrutural — Composite: tratar colunas e cartões por uma interface comum.
– Comportamental — State: restringir ações conforme o estado do cartão.

Cenários de teste

T3.1 — Mover cartão entre colunas

T3.2 — Validar cartão incompleto

T3.3 — Sincronização e conflito de edição

T3.4 — State e Composite

4. Gerenciador de quizzes

Design patterns a demonstrar

– Criacional — Abstract Factory: criar famílias consistentes de perguntas e opções.
– Estrutural — Adapter: converter formatos de importação para o modelo interno.
– Comportamental — Strategy: selecionar método de pontuação/correção.

Cenários de teste

T4.1 — Responder quiz e ver resultado

T4.2 — Validar quiz malformado

T4.3 — Importação e submissão pela API

T4.4 — Abstract Factory e Strategy de pontuação

5. Fórum de discussão

Design patterns a demonstrar

– Criacional — Factory Method: criar tópico, resposta ou anúncio.
– Estrutural — Decorator: adicionar marcações (fixado, fechado) sem alterar tópico base.
– Comportamental — Chain of Responsibility: aplicar etapas de moderação.

Cenários de teste

T5.1 — Criar tópico e responder

T5.2 — Rejeitar conteúdo inválido

T5.3 — Permissão e persistência

T5.4 — Cadeia de moderação e Decorator

6. Sistema de votação para enquetes

Design patterns a demonstrar

– Criacional — Factory Method: criar enquete de escolha única ou múltipla.
– Estrutural — Proxy: controlar elegibilidade e limitar voto duplicado.
– Comportamental — Observer: atualizar resultados após voto válido.

Cenários de teste

T6.1 — Votar e atualizar resultados

T6.2 — Validar seleção e prazo

T6.3 — Impedir voto duplicado via API

T6.4 — Proxy e Observer

7. Gerenciador de banco de dados JSON

Design patterns a demonstrar

– Criacional — Builder: montar documentos JSON com campos e tipos definidos.

– Estrutural — Adapter: uniformizar leitura/gravação entre fontes ou formatos.

– Comportamental — Command: encapsular operações com execução e desfazer controlado.

Cenários de teste

T7.1 — Criar, editar e consultar documento

T7.2 — JSON inválido e tipos incompatíveis

T7.3 — Atomicidade de gravação e consulta

T7.4 — Builder, Adapter e Command

8. Gerenciador de notas e atividades de alunos

Design patterns a demonstrar

– Criacional — Factory Method: criar atividade, avaliação e lançamento de nota.
– Estrutural — Facade: reunir operações de turma, aluno e boletim.
– Comportamental — Strategy: calcular médias com critérios configuráveis.

Cenários de teste

T8.1 — Registrar nota e atualizar boletim

T8.2 — Validar nota, peso e vínculo

T8.3 — Persistência e controle de acesso

T8.4 — Facade e Strategy de média

9. Gerenciador de avaliações de jogos

Design patterns a demonstrar

– Criacional — Builder: construir avaliação com nota, texto e metadados opcionais.
– Estrutural — Proxy: restringir edição/exclusão ao autor ou moderador.
– Comportamental — Observer: atualizar nota agregada após publicação/moderação.

Cenários de teste

T9.1 — Publicar avaliação

T9.2 — Validar nota e comentário

T9.3 — Autorização de edição e atualização agregada

T9.4 — Builder, Proxy e Observer

10. Organizador de imagens

Design patterns a demonstrar

– Criacional — Abstract Factory: criar leitores/miniaturas compatíveis com formatos suportados.

– Estrutural — Flyweight: compartilhar metadados/recursos comuns em miniaturas.

– Comportamental — Strategy: trocar algoritmo de ordenação, filtro ou geração de miniaturas.

Cenários de teste

T10.1 — Importar e visualizar imagens

T10.2 — Arquivo inválido e caminho ausente

T10.3 — Upload, metadados e download

T10.4 — Flyweight e Strategy

11. Organizador de projetos

Design patterns a demonstrar

– Criacional — Builder: montar projetos com propriedades e tarefas opcionais.

– Estrutural — Composite: representar projeto, fases e tarefas hierarquicamente.

– Comportamental — Mediator: coordenar colaboração e atualizações entre módulos/usuários.

Cenários de teste

T11.1 — Criar projeto e hierarquia de tarefas

T11.2 — Validar dependências e campos

T11.3 — Colaboração e persistência

T11.4 — Composite e Mediator

12. Interface de e-commerce

Design patterns a demonstrar

– Criacional — Abstract Factory: criar famílias de componentes/fluxos por meio de pagamento.

– Estrutural — Adapter: integrar gateway externo com interface interna de pagamento.

– Comportamental — State: controlar estados e transições do pedido.

Cenários de teste

T12.1 — Fluxo de compra pela interface

T12.2 — Validar quantidade, endereço e preço

T12.3 — Pagamento e atualização cliente-servidor

T12.4 — State e Adapter/Abstract Factory